

DIRETRIZES DO TRABALHO - UNIDADE I

Disciplina: DIM0501 - Boas Práticas de Programação

Tema: Aplicação dos conceitos de boas práticas em cenário simulado

Semestre: 2026.1

1. OBJETIVO

Este trabalho tem como objetivo consolidar, de forma prática em um cenário simulado, os conceitos abordados na disciplina, com foco em:

- Qualidade de código
- Legibilidade e organização
- Redução de dívida técnica
- Aplicação de boas práticas profissionais

Os alunos irão trabalhar sobre um **código inicial propositalmente mal estruturado**, realizando melhorias progressivas com base nos conteúdos estudados.

Como resultado, os grupos devem demonstrar, através da apresentação de um relatório simplificado, os problemas encontrados e as estratégias utilizadas para resolverem esses problemas (ex.: refatoração, modularização, programação defensiva e documentação).

2. FORMAÇÃO DOS GRUPOS

- O trabalho deve ser realizado em grupos de **no máximo 5 (cinco)** alunos.
- Todos os membros devem participar ativamente do desenvolvimento e da documentação.

3. DESCRIÇÃO DA ATIVIDADE

Cada grupo deverá executar a atividade proposta tendo como base no seguinte material apresentado:

- O **cenário de aplicação** (problema contextualizado)
- O **código inicial com problemas**, escrito em 3 linguagens diferentes (Python, C/C++ e Java).

Importante: Cada equipe deverá **trabalhar em apenas um dos códigos**, selecionado de acordo com a linguagem de preferência.

3.1 Cenário do Problema

Você faz parte de uma equipe que recebeu a responsabilidade de executar a manutenção de um “Sistema de Monitoramento de Fake News e Qualidade da Informação”. O sistema permite:

- Inserir textos de notícias
- Classificá-las como **confiável**, **duvidosa** ou **falsa**
 - Critérios de análise:
 - Ausência de fonte
 - Uso de linguagem sensacionalista
 - Tamanho do texto
- Listar as notícias cadastradas

No entanto, o código atual, apesar de simples, foi desenvolvido sem boas práticas e apresenta diversos problemas de qualidade, dificultando sua manutenção e evolução.

Seu objetivo é **melhorar esse sistema sem reescrevê-lo do zero**.

3.2 Código Inicial

Os códigos estão disponíveis em um [projeto no GitHub](#) e podem ser acessados a partir dos links abaixo:

Versão em C++:

<https://github.com/marcusterraufnr/Disciplinas/tree/main/2026/1/BPP/Unidades/1/Trabalho/C%2B%2B>

Versão em Java:

<https://github.com/marcusterraufnr/Disciplinas/tree/main/2026/1/BPP/Unidades/1/Trabalho/Java>

Versão em Python:

<https://github.com/marcusterraufnr/Disciplinas/tree/main/2026/1/BPP/Unidades/1/Trabalho/Python>

O grupo deve escolher **apenas uma das versões** para realizar o trabalho.

4. REQUISITOS DO TRABALHO

O trabalho deverá ser dividido em **cinco** etapas fundamentais:

Etapla 1. Identificar Problemas

Os grupos devem analisar o código e identificar problemas relacionados a:

- Legibilidade
- Organização
- Nomes inadequados
- Código duplicado
- Falta de validação
- Comentários inadequados ou inúteis
- “Code smells” básicos

Etapla 2. Refatorar Nomes e Funções

Deve-se reestruturar internamente o código-fonte sem alterar o comportamento externo, buscando:

- Melhorar nomes de variáveis, funções e estruturas
- Quebrar funções grandes
- Garantir que cada função tenha uma responsabilidade clara
- Tornar o código mais legível e reutilizável

Etapla 3. Organizar em Módulos

A organização modular deve ter por objetivo:

- Separar o código em partes com responsabilidades distintas (ex: modelo, lógica, interface)
- Melhorar a estrutura geral do projeto

Etapla 4. Adicionar Validações

A equipe precisa aplicar alguns conceitos básicos de Programação Defensiva, tais como:

- Validar entradas do usuário
- Tratar erros simples
- Evitar comportamentos inesperados

Etapla 5. Melhorar Documentação

A análise da documentação é parte importante do trabalho e deve buscar:

- Remover comentários inúteis ou redundantes
- Adicionar comentários úteis quando necessário
- Criar documentação básica:
 - Docstrings
 - Arquivo README

5. ENTREGÁVEIS

1. Código-Fonte Refatorado

- Deve ser disponibilizado em um repositório no [GitHub](#)
- Deve conter:
 - Histórico das alterações feitas nos arquivos do projeto (commits)
 - Organização clara do projeto

2. Relatório Técnico Simplificado (Formato PDF)

O grupo deverá entregar um relatório técnico em formato PDF descrevendo:

- Problemas identificados
- Soluções aplicadas
- Justificativas das decisões

O relatório deve seguir a estrutura apresentada no modelo no final deste documento.

6. CRITÉRIOS DE AVALIAÇÃO (PESO 10.0)

Critério	Peso
Identificação de problemas	20%
Qualidade da refatoração	25%
Organização do código	15%
Programação Defensiva (Validações e robustez)	10%
Documentação do código e do projeto	10%
Relatório Técnico	20%

7. CRONOGRAMA

Etapa	Data prevista
Definição das Diretrizes do Trabalho	18/04/2026
Produção do Trabalho	18/04/2026 a 08/05/2026
Entrega da versão final do código fonte refatorado e do Relatório Técnico Simplificado.	08/05/2026

8. ORIENTAÇÕES GERAIS

Segue abaixo algumas orientações importantes a serem adotadas na execução do trabalho:

- Escolha **apenas uma** versão entre as linguagens existentes;
- **Não é permitido** reescrever o sistema do zero;
- O foco é **melhorar o código existente**;
- O código deve manter o **funcionamento original**;
- **Evite** complexidade desnecessária;

Dica Final

Se outra pessoa conseguir entender seu código e seu relatório sem explicação adicional, vocês fizeram um bom trabalho.

Modelo de Relatório Técnico

Disciplina: DIM0501 - Boas Práticas de Programação

Tema: Aplicação dos conceitos de boas práticas em cenário simulado

Semestre: 2026.1

1. Identificação do Grupo

Integrante 1:

Integrante 2 (se houver):

Integrante 3 (se houver):

Integrante 4 (se houver):

Integrante 5 (se houver):

Linguagem utilizada: (Python, C/C++ ou Java)

Link do repositório Git:

2. Descrição do Sistema

Descreva brevemente:

- O que o sistema faz
- Qual era o objetivo do código original
- Em que contexto ele poderia ser utilizado

3. Problemas Identificados

Liste e explique os principais problemas encontrados no código original.

3.1 Problemas de Legibilidade

Exemplo:

- Nomes de variáveis e funções pouco descritivos
- Dificuldade de compreensão do fluxo do código

Trecho de código (exemplo):

<cole aqui um trecho do código original>

3.2 Problemas de Organização

Exemplo:

- Funções com múltiplas responsabilidades
- Falta de separação em módulos

3.3 Código Duplicado ou Design Ruim

Exemplo:

- Repetição de lógica
- Estruturas inadequadas

3.4 Falta de Validação

Exemplo:

- Entradas não verificadas
- Possibilidade de erro em tempo de execução

3.5 Problemas de Documentação

Exemplo:

- Comentários inúteis
- Ausência de explicações relevantes

4. Estratégias de Solução

Explique como os problemas foram resolvidos.

4.1 Refatoração

- O que foi renomeado
- Como as funções foram reorganizadas

4.2 Modularização

- Como o código foi dividido
- Quais módulos foram criados

4.3 Programação Defensiva

- Quais validações foram adicionadas
- Como erros passaram a ser tratados

4.4 Documentação

- O que foi documentado
- Que tipo de comentários foram utilizados
- O que foi incluído no README

5. Exemplos de Melhoria (Antes vs Depois)

Apresente pelo menos **2 exemplos claros** de melhoria.

Exemplo 1

Antes:

<trecho original>

Depois:

<trecho refatorado>

Explicação:

Explique o que foi melhorado e por quê.

Exemplo 2

Antes:

<trecho original>

Depois:

<trecho refatorado>

Explicação:

Explique o que foi melhorado e por quê.

6. Organização Final do Código

Descreva a estrutura final do projeto:

Exemplo:

```
projeto/  
├── modelo  
├── servico  
├── interface  
└── main
```

Explique brevemente o papel de cada parte.

7. Conclusão

Responda:

- O que o grupo aprendeu com o trabalho
- Quais foram as maiores dificuldades
- Como as boas práticas impactaram o código